

CS3002E Compiler Design

Monsoon 2025

Lecture #20

Syntax Directed Translation: Introduction

Department of Computer Science & Engineering
NIT Calicut

Topics

- Syntax Directed Translation - Introduction
- Syntax Directed Definition (SDD)
 - Attribute Evaluation
 - Dependency Graph
 - Synthesized and Inherited attributes

Syntax Directed Translation

Syntax Directed Translation is a grammar-oriented compiling technique.

- translation is guided by Syntax - Context Free Grammar rules help guide the translation.
 - e. g. generate Abstract Syntax Tree (AST) representation of the program based on the syntactic structure of the program.
- translation is done by attaching actions to grammar productions.

Example: Expression evaluation

Production	Action Required
$E \rightarrow num$	Set value of E as the lexical value of num
$E \rightarrow E_1 + E_2$	Compute value of E as value of E_1 + value of E_2

- An action associated with each production.
- Set of actions translates the given expression to a value.

Syntax Directed Translation

Translation guided by syntax:

- Associate *attributes* with grammar symbols (each symbol represents a language construct)
- Attach *semantic rules* to grammar productions
- Rules are executed when the production is used during syntax analysis

Example: Expression evaluation

Production	Action Required
$E \rightarrow num$	Set value of E as the lexical value of num $E.val = num.lexval$
$E \rightarrow E_1 + E_2$	Compute value of E as value of E_1 + value of E_2 $E.val = E_1.val + E_2.val$

An attribute associated with each grammar symbol.

$E.val$ denotes the attribute val associated with E .

$num.lexval$ - lexical value associated with token **num**.

Syntax Directed Definition (SDD)

Syntax Directed Definition (SDD): A notation for specifying Syntax Directed Translation.

- Context Free Grammar together with **attributes** and **rules**
- **Attributes** associated with each grammar symbol
 - $X.a$ denotes the value of a associated with grammar symbol X .
 - attributes can be numbers, types, strings, references etc.
- **Rules** associated with each production. Attribute values are computed as per the rules.
- Attribute values associated with terminals (*lexical values*) are obtained during lexical analysis.

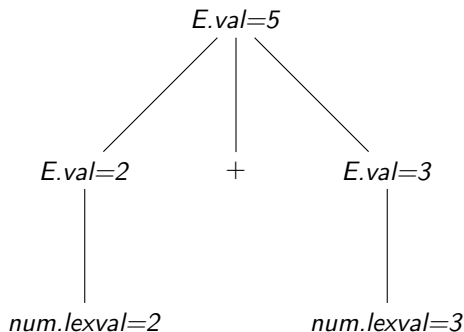
SDD: Example

PRODUCTION	SEMANTIC RULE
$E \rightarrow num$	$E.val = num.lexval$
$E \rightarrow E_1 + E_2$	$E.val = E_1.val + E_2.val$

- Specifies the evaluation of an expression based on the values of its subexpressions.
- Action associated with a production can be executed when the parser uses the production.
- The attribute *lexval* associated with **num** is assumed to be set during lexical analysis

Annotated Parse Tree

Parse Tree annotated with attribute value at each node



Annotated Parse Tree for $2+3$

SDD: example

Production	Semantic Rules
$S \rightarrow E$	$print(E.val)$
$E \rightarrow E_1 + E_2$	$E.val = E_1.val + E_2.val$
$E \rightarrow \mathbf{num}$	$E.val = \mathbf{num.lexval}$

Note the rule $print(E.val)$ attached to the first production, for printing the value of the expression

SDD: example

Production	Semantic Rules
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow \mathbf{num}$	$T.val = \mathbf{num.lexval}$

Draw the annotated Parse Tree for 2+3

Exercise

Given the grammar:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow \mathbf{num}$$

Write SDD to evaluate a given expression. Assume *num.lexval* is set by the lexical analyzer. Draw the annotated parse tree for $1 + 2 * 3$.

Synthesized / Inherited attributes

- **Synthesized attribute:** Attribute value at node N defined only in terms of attribute values at the children of N and at N itself.
- **Inherited attribute:** Attribute value at node N is defined only in terms of attribute values at N 's parent, at N itself and N 's siblings.

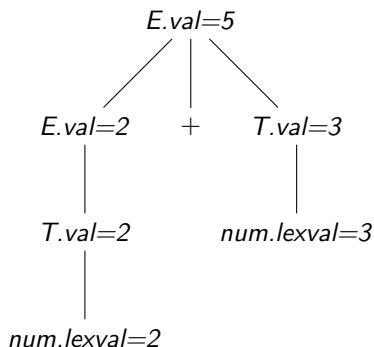
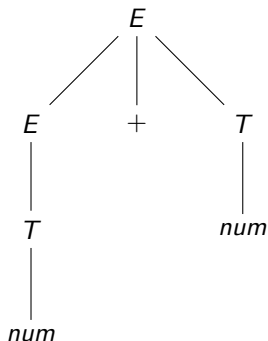
Attribute Evaluation

Possible approaches:

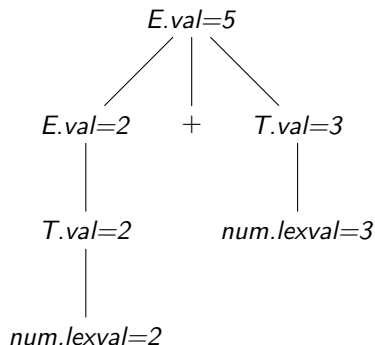
- Construct a Parse Tree, visit the nodes of the tree and compute the attribute values.
- Evaluate the attributes during parsing, without explicitly building a parse tree.

Attributes: Order of evaluation

Assume the parse tree is already built. Give a possible evaluation order of attributes in order to construct the annotated parse tree.



Attributes: Dependency



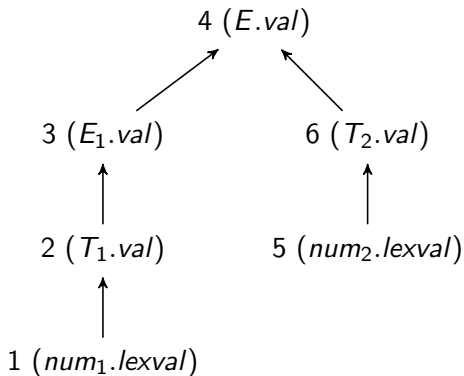
- Attribute at a node is dependent on the attributes of its children.
- Visit the nodes in a bottom-up order, evaluating attributes at each node.

Dependency Graph

Dependency Graph depicts the dependency among attributes.

- for each grammar symbol X , the graph has a node for each attribute associated with X .
- a directed edge from attribute $X.a$ to attribute $Y.b$ to indicate that $Y.b$ is dependent on $X.a$ (value of $X.a$ is needed to compute value of $Y.b$).
 - for a synthesized attribute, dependency edge goes from child to parent.
 - for an inherited attribute, dependency edge goes from parent to child or from sibling to sibling.
- dummy attributes corresponding to the application of functions.

Dependency Graph: example

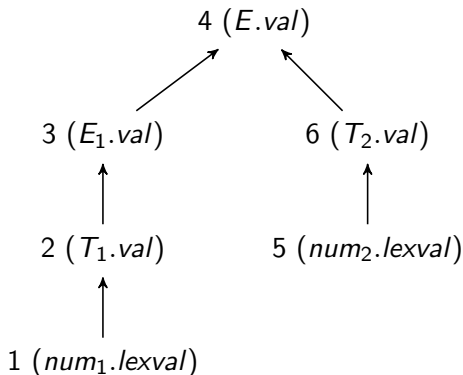


Note: 1, 2, ..., 6 are the nodes of the dependency graph.

Order of evaluation of attributes

- If there is an edge from attribute $X.a$ to attribute $Y.b$, evaluate $X.a$ before evaluating $Y.b$.
- If the dependency graph has no cycles, attributes can be evaluated in a *topological sort order* of the graph.

Dependency Graph: Evaluation order



- Since the graph is acyclic, the attributes can be evaluated according to a *topological sort order* of the nodes.
- A possible order: 1, 2, 3, 5, 6, 4.

Symbol Table

Symbol Table: A data structure that maintains information regarding the source program.

- One entry for each name (*symbol*) - names of variables, symbolic constants, labels, procedure/function names etc., with values of its attributes.

For example, the entry for a variable name may contain the name, type and relative address of that variable.

- Stores information collected during initial phases of *Analysis*, that will be required for later stages of compilation.

Inherited Attributes: Example - SDD for setting types in Symbol Table

Production	Semantic Rules
$D \rightarrow T L$	$L.type = T.type$
$T \rightarrow \text{int}$	$T.type = int$
$T \rightarrow \text{float}$	$T.type = float$
$L \rightarrow L_1, \text{id}$	$L_1.type = L.type$ $addType(\text{id.entry}, L.type)$
$L \rightarrow \text{id}$	$addType(\text{id.entry}, L.type)$

Attribute $L.type$ is *inherited*.

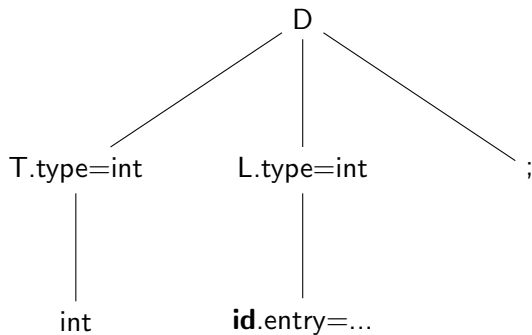
- Semantic rule $L.type = T.type$ associated with $D \rightarrow T L$
 $L.type$ is dependent on the attribute at its sibling node.
- Semantic rule $L_1.type = L.type$ associated with $L \rightarrow L_1, \text{id}$
 $L_1.type$ is dependent on the attribute at its parent node.

Inherited Attributes: Example - SDD for setting types in Symbol Table

Production	Semantic Rules
$D \rightarrow T L$	$L.type = T.type$
$T \rightarrow \mathbf{int}$	$T.type = int$
$T \rightarrow \mathbf{float}$	$T.type = float$
$L \rightarrow L_1, \mathbf{id}$	$L_1.type = L.type$ $addType(\mathbf{id}.entry, L.type)$
$L \rightarrow \mathbf{id}$	$addType(\mathbf{id}.entry, L.type)$

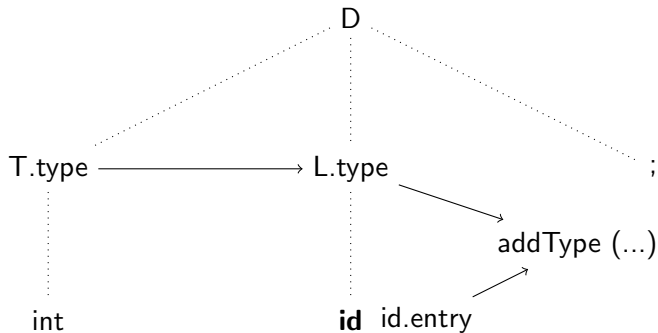
- $\mathbf{id}.entry$ is a pointer to the Symbol Table entry for $\mathbf{id}$. For each identifier, the value of $entry$ is assumed to be obtained from the Lexical Analyzer.
- $addType(id.entry, L.type)$ is invoked to set the value of type as $L.type$ in the Symbol Table entry pointed to by $id.entry$.

Annotated Parse Tree

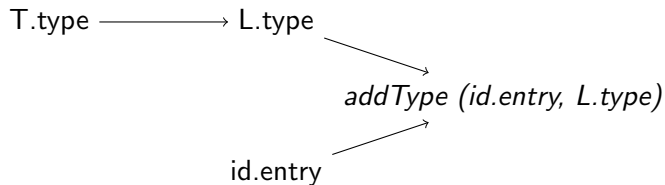


Annotated Parse Tree for `int id;`

Dependency among attributes



Dependency graph



SDD: S-attributed / L-attributed

S-attributed SDD: involves only synthesized attributes.

L-attributed SDD: attributes can be synthesized or inherited, but with the restrictions that dependency graph edges between attributes of symbols in a production body go from left to right¹.

¹those interested may see section 5.2.4 of ALSU book for a precise definition

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Chapter 2: sections 2.3, Chapter 5: section 5.1, 5.2 (upto 5.2.3)